

ЗАДАНИЕ 1.

1. Зафиксируйте в таблице: метод, URL, код ответа, размер данных, время загрузки.

![[Pasted image 20260909190952.png]]

2.3. Детальный разбор одного запроса:

- метод; GET
- URL и параметры; - <https://jsonplaceholder.typicode.com/posts> - Параметров нет
- заголовки (Response Headers, Request Headers);

![[file:///C:/Users/engineer/AppData/Local/Temp/msohtmlclip1/01/clip_image004.png]]

![[file:///C:/Users/engineer/AppData/Local/Temp/msohtmlclip1/01/clip_image006.png]]

- тело ответа (Response / Preview);

![[file:///C:/Users/engineer/AppData/Local/Temp/msohtmlclip1/01/clip_image008.png]]

![[file:///C:/Users/engineer/AppData/Local/Temp/msohtmlclip1/01/clip_image010.png]]

- код ответа;

200 OK

- время выполнения.

![[file:///C:/Users/engineer/AppData/Local/Temp/msohtmlclip1/01/clip_image012.png]]

![[file:///C:/Users/engineer/AppData/Local/Temp/msohtmlclip1/01/clip_image014.png]]

![[file:///C:/Users/engineer/AppData/Local/Temp/msohtmlclip1/01/clip_image016.png]]

![[file:///C:/Users/engineer/AppData/Local/Temp/msohtmlclip1/01/clip_image018.png]]

![[file:///C:/Users/engineer/AppData/Local/Temp/msohtmlclip1/01/clip_image020.png]]

ЗАДАНИЕ №2

- GET /posts (все посты);
- GET /posts/1 (один пост).

Сделайте выводы по:

- размеру ответа;

![[file:///C:/Users/engineer/AppData/Local/Temp/msohtmlclip1/01/clip_image022.png]]

![[file:///C:/Users/engineer/AppData/Local/Temp/msohtmlclip1/01/clip_image023.png]]

- времени загрузки;

![[file:///C:/Users/engineer/AppData/Local/Temp/msohtmlclip1/01/clip_image025.png]]

![[file:///C:/Users/engineer/AppData/Local/Temp/msohtmlclip1/01/clip_image027.png]]

- количеству записей.

![[file:///C:/Users/engineer/AppData/Local/Temp/msohtmlclip1/01/clip_image029.png]]

2. сколько объектов вернулось; 1 объект

100 объектов

3. размер ответа;

4. время загрузки.

Сделайте вывод, как параметры помогают **уменьшить объём данных** – чем больше фильтров, тем меньше объём данных

ЗАДАНИЕ №3

- как меняется время отклика;

- как браузер параллелит запросы;

Браузер выполняет вопросы одновременно.

Второй вопрос выполняет с погрешностью в миллисекунду.

браузер не ждёт, пока завершится первый запрос, чтобы начать второй

- нет ли избыточных запросов.

1 — 77 В

2 — 53 В

3 — 53 В

Все ответы — **микроскопические** (меньше 100 байт).

2.7. Заголовки и ответы

Во вкладке **Headers** выпишите не менее 5 ключевых заголовков, например:

Content-Type — указывает тип передаваемых данных (например, text/html для веб-страниц или application/json для данных), чтобы браузер знал, как правильно отобразить информацию.

Cache-Control — управляет кэшированием. Он говорит браузеру, можно ли сохранять страницу или файл на компьютере и как долго их использовать повторно без обращения к серверу.

ETag (Entity Tag) — уникальный цифровой код конкретной версии файла. Если файл не менялся, код остается прежним, что помогает загружать данные быстрее.

Date — показывает точное время и дату, когда сервер отправил этот ответ.

Server — содержит информацию о программном обеспечении самого веб-сервера (например, *Nginx* или *Apache*).

(HTTP/1.1 200 OK | | | | Content-Type: application/json ← Формат данных | | | | Content-Length: 77 ← Размер ответа | | | | Cache-Control: max-age=43200 ← Кэшировать | | | | ETag: "W/6b80-..." ← Версия файла | | |)

(GET /posts/1 HTTP/1.1
 | | | | | Host: jsonplaceholder.typicode.com ← Куда
 | | | | | User-Agent: Mozilla/5.0 ← Кто (браузер)
 | | | | | Accept: application/json ← Что хочу
 | | | | | Accept-Encoding: gzip, deflate ← Как сжимать
 | | | Connection: keep-alive ← Держать связь

7. Сравните вкладки **Preview** и **Response**:

Вкладки **Preview** (Просмотр) и **Response** (Ответ) находятся в панели разработчика браузера (вкладка Network) и показывают данные, пришедшие от сервера.

чем они отличаются по представлению данных

Главное отличие — **формат отображения**

в каких случаях удобнее использовать Preview, а в каких — Response.

	Content-Type	Cache-Control	ETag	Date	Server
https://jsonplaceholder.typicode.com/posts	application/json; charset=utf-8	max-age=43200	W/"6b80-Ybsq/K6GwwqrYkAsFqxqDXGC7DoM"	Wed, 09 Sep 2026 11:57:49 GMT	cloudflare

ЗАДАНИЕ 4

2.9. Работа с API через Postman

1. Перейдите на сайт:

<https://www.postman.com/>

1. Выполните:

2. GET /posts ![Pasted image 20260909151621.png]

3. GET /posts/1 ![Pasted image 20260909151640.png] Сравните ответы.

4. Выполните **POST** /posts с JSON-телом:

```

{
  "title": "Пост через Postman",
  "body": "Текст поста",
  "userId": 2
}
```

![Pasted image 20260909185927.png]

1. Выполните:

2. **PUT** /posts/1 — полное обновление ресурса; ![Pasted image 20260909190104.png]

3. **PATCH** /posts/1 — частичное обновление (изменить только один-два поля). ![Pasted image 20260909190140.png]

Объясните, чем отличаются результаты.

1. Выполните **DELETE** /posts/1 и зафиксируйте:

2. код ответа;

3. тело ответа (если есть). ![[Pasted image 20260909190228.png]]